Assignment 2: Text Processing and Classification
using Apache Spark (RDDs, DataFrames, and ML Pipelines)

Group 48 – Data-Intensive Computing, TU Wien, SS 2026
Aaradhaya Arora (12534787) Ekaterina Dobrynina (12332276) Gopinath Hariharasudhan (52202173)
Serhii Slutu (12537831) Sofia Zubkova (12534780)

May 19, 2026

AI disclosure. We used various AI tools (e.g., ChatGPT, DeepL) as a support tool while working on this assignment. Its use was limited to text polishing (grammar, wording) and to debugging help when we were stuck.

1 Introduction

In Assignment 1, the fundamental principles of text processing using MapReduce were explored. In this assignment, the focus is on leveraging the capabilities of Apache Spark to manage large text corpora with enhanced efficiency and a more contemporary API. The objective is to port the chi-square term selection task to Spark and extend it into a full text classification pipeline. The assignment is divided into three components:

1. **Part 1 (RDDs):** Chi-square feature selection pipeline from Assignment 1 is to be re-implemented using the Spark Resilient Distributed Dataset (RDD) API
2. **Part 2 (DataFrames and ML Pipelines):** Feature extraction process is to be converted to a Spark Machine Learning Pipeline using DataFrames, with integration of tokenization, stopword removal, Term Frequency-Inverse Document Frequency calculation, and Chi-Square selection
3. **Part 3 (Text Classification):** Training of a Support Vector Machine (SVM) multi-class classifier on the extracted features is undertaken, with the parameters being optimised via grid search and the F1 performance evaluated

All development and evaluation were conducted on the development set (`reviews_devset.json`) to optimize resource usage on the shared cluster, with a sampled subset for the hyperparameter grid search due to the computational complexity of the SVM.

2 Problem Overview

Part 1: RDD Implementation. The same preprocessing steps are performed as in Assignment 1, namely lowercasing, splitting by specific delimiters, and filtering out stopwords and single-character tokens. The task at hand necessitates the calculation of chi-square values for each term within each category, with the objective of extracting the top 75 terms in each category, in addition to the globally joined dictionary. The objective of this study is to ascertain whether it is possible to obtain equivalent results as MapReduce, but utilising the Spark RDD paradigm.

Part 2: Spark ML Pipeline. In place of manual map-reduce steps, an automated transformation pipeline is constructed utilising Spark's MLlib. It is imperative that built-in functions are capable of handling tokenization (at whitespaces, digits, and special delimiters), stopword removal, term frequency-inverse document frequency (TF-IDF) extraction, and top-2000 feature selection globally via Chi-Square. The resultant output is a list comprising the 2000 selected features.

Part 3: Text Classification. The third part of this study concerns text classification. In order to employ a multi-class SVM classifier, it is necessary to first train the model on TF-IDF features. It is imperative that the reviews are correctly classified into their respective product categories. The application of vector length normalization (L2 norm) prior to classification is a fundamental aspect of the methodology. In addition, a hyperparameter grid search must be implemented, encompassing the exploration of feature subset sizes, regularization parameters, standardizations, and maximum iterations, with the utilisation of cross-validation over training, validation, and test splits.

3 Methodology and Approach

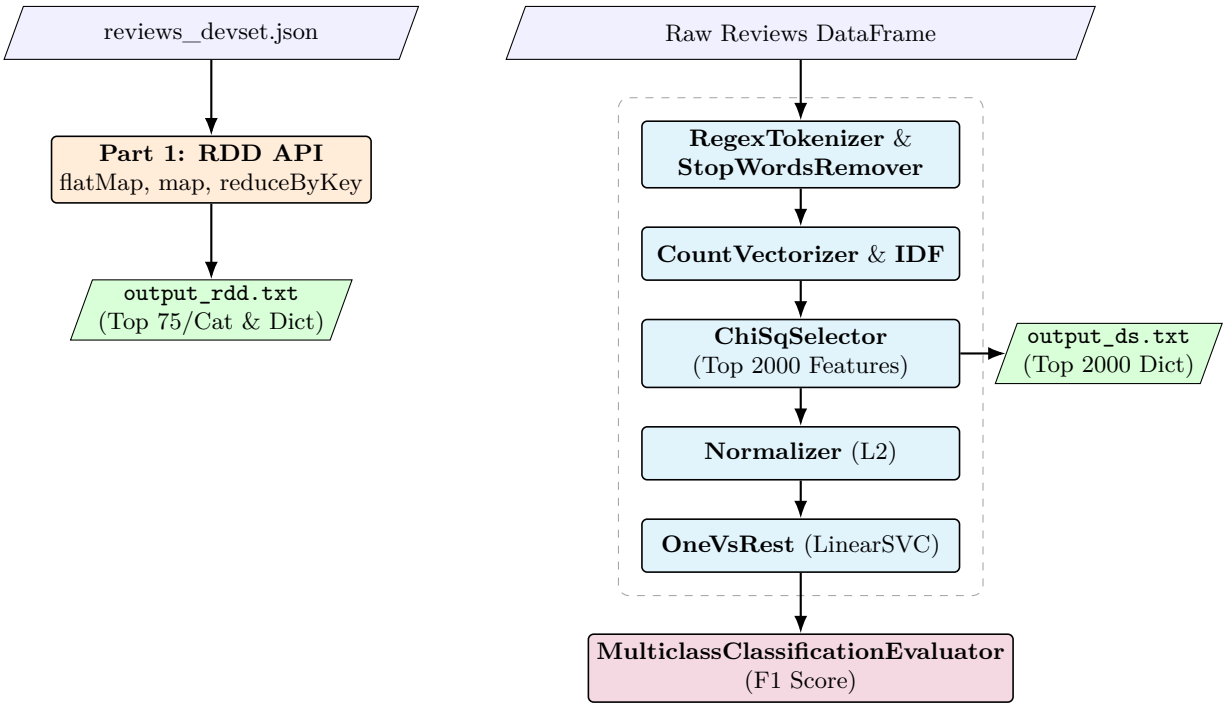

Figure 1: Overview of the Spark pipelines implemented in Parts 1, 2, and 3

3.1 Part 1: RDDs

The procedure involved in this assignment consists of loading the JSON file into an RDD, extracting the reviewText and category, and performing the custom deduplication mapping in the same way as in Assignment 1. The application of the flatMap and reduceByKey functions enables the calculation of document frequencies. The intermediate total counts (e.g., N and n_c) are cached, and then broadcast as a dictionary to worker nodes. This enables efficient local calculations of the chi-squared statistic.

3.2 Part 2: DataFrames and Pipelines

The transition from RDDs to DataFrames is a significant development in the field. The pipeline is comprised of the following elements:

- `RegexTokenizer`: Text is divided into whitespace, digits and the specified punctuation marks
- `StopWordsRemover`: Removes default English stopwords
- `CountVectorizer` and `IDF`: Computes TF-IDF vectors for each document

- `ChiSqSelector`: Selects the top 2000 features indicative of category labels across the entire corpus

3.3 Part 3: Text Classification

The pipeline is augmented with an L2 Normalizer to scale the features. In the context of multi-class categorisation, a OneVsRest wrapper is employed around a LinearSVC. In order to evaluate the model within a realistic timeframe and resource scope, a sample of 10,000 instances was drawn from the dataset. The data was divided into three sets: Training (approximately 71%), Validation (14%), and Test (15%). The hyperparameters (top features, regularisation, standardisation, and maximum iterations) were tuned using a ParamGridBuilder.

4 Results

4.1 Part 1 Observations

The execution yielded the file entitled `output_rdd.txt`. A comparison of this output with that of Assignment 1 (see `output.txt`) reveals that the top 75 terms selected per category, the resulting chi-squared values, and the globally joined dictionary are essentially identical. This finding serves to substantiate the hypothesis that the RDD implementation is capable of computing statistical values with perfect accuracy. However, the in-memory computation model employed by Spark resulted in a substantial acceleration of the shuffle-and-sort phases, in comparison to the disk-heavy MapReduce equivalent.

4.2 Part 2 Observations

The ML pipeline generated the output file, which is denoted as `output_ds.txt` and consists of 2000 top terms across the dataset. The terms selected here demonstrate a high degree of overlap with those selected manually in Assignment 1. However, minor discrepancies have been observed. This discrepancy can be attributed primarily to the fact that Spark's integrated ChiSqSelector computes the top K features globally based on dependence against the label column. This approach differs fundamentally from our manual per-category top-75 slice strategy. Furthermore, it should be noted that Spark's RegexTokenizer and StopWordsRemover manage edge cases (e.g., empty tokens, varying stopword lists) in a manner that differs from a manual regex approach.

4.3 Part 3 Performance

The dataset was distributed as follows: The total number of training instances is 7,104, the number of validation instances is 1,432, and the number of test instances is 1,464. An evaluation was conducted of a focused hyperparameter configuration grid with the objective of preserving cluster compute:

- **Top Features:** 500
- **SVM Regularization Param (`regParam`):** 0.1
- **Standardization:** True
- **Max Iterations:** 20

The trained SVM model yielded excellent results on the extracted feature space:

- **Validation F1 Score:** 0.9307
- **Test F1 Score:** 0.9356

The model's accuracy is confirmed by sample predictions. Reviews discussing "Sword of Truth" or "A book every mom should read" were correctly predicted as index 0.0 (Book category), while reviews about game performance issues were mapped to index 1.0 (Apps_for_Android). The elevated F1 score signifies that a subset of 500 TF-IDF terms, selected via the Chi-Square method, offers an exceptionally separable hyperplane for the Linear SVC.

5 Conclusions

The implementation of a scalable text processing and classification pipeline utilising Apache Spark was successfully executed. In the initial part of this assignment, the accuracy of the RDD API was validated through the successful reproduction of the MapReduce output from Assignment 1. In Parts 2 and 3, the Spark's DataFrame MLlib API was leveraged to efficiently construct a transformation pipeline containing tokenization, TF-IDF scaling, Chi-Square selection, and a One-vs-Rest Support Vector Machine classifier.

The findings of our grid-search approach and evaluations demonstrated that Spark ML pipelines drastically simplify robust feature engineering. The resulting SVM classifier achieved a highly competitive F1 score of 0.935 on the test set, thus proving the efficacy of the integration of TF-IDF and Chi-Square at extracting discriminative semantic signals from the review texts.